GUÍA DEMOSTRATIVA DE LABORATORIO COMPLETA

Programa de Formación: Tecnólogo en Análisis y Desarrollo de Software (ADSO)

Competencia 2: Elaboración de la Propuesta Técnica del Software

Actividad de Aprendizaje: AA2 — Verificación y Auditoría de APIs RESTful con Thunder Client

Caso de Estudio Demostrativo: Sistema de Gestión de Biblioteca Pública (Módulo Completo de Libros)

Stack Tecnológico: Python 3.x, FastAPI, Uvicorn, Visual Studio Code, Thunder Client

Duración: 6 Horas

1. PRESENTACIÓN Y OBJETIVOS DEL LABORATORIO

Esta guía demostrativa le servirá al instructor para dirigir la **Sesión Guiada de Laboratorio (Live Coding)**. A través de un servicio backend completo construido en FastAPI para una Biblioteca Pública, el aprendiz aprenderá a verificar la calidad, seguridad, tipos de datos y códigos de estado HTTP de una API RESTful antes de integrarla con las aplicaciones Web (Angular) y Móviles (Kotlin/Flutter).

Objetivos específicos:

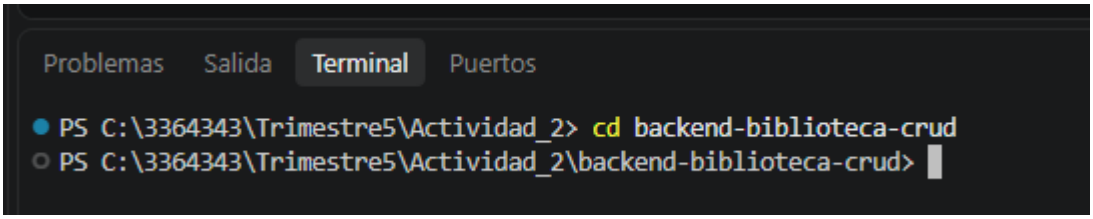
1. Construir e iniciar un servicio API RESTful en **FastAPI** implementando el ciclo **CRUD completo** con validación de datos mediante **Pydantic**.
2. Configurar la extensión **Thunder Client** en VS Code utilizando el almacenamiento local (**Save To Workspace**).
3. Estructurar y ejecutar peticiones HTTP (GET, POST, PUT, DELETE), enviando encabezados (Headers) y cuerpos de mensaje en formato JSON.
4. Auditar la calidad de la API evaluando los códigos de estado HTTP (200 OK, 201 Created, 400 Bad Request, 404 Not Found) y los tiempos de latencia.
5. Exportar la colección de pruebas e integrarla al repositorio de **GitHub**.

2. CONFIGURACIÓN DEL ENTORNO Y CÓDIGO BACKEND (DESDE CERO)

Paso 1: Crear la estructura del proyecto

Abre la terminal de tu sistema operativo o la terminal integrada de VS Code (Ctrl + ñ) y ejecuta los siguientes comandos para crear la carpeta de trabajo:

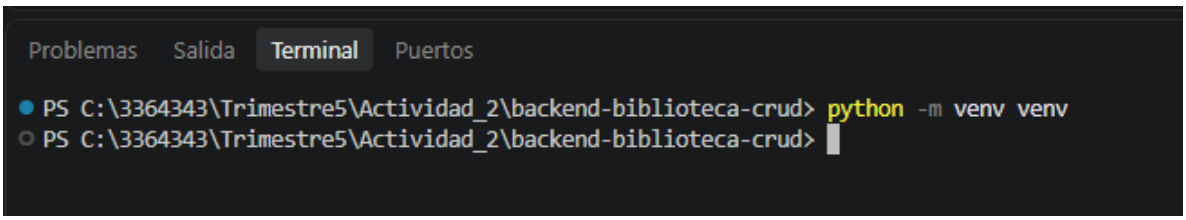


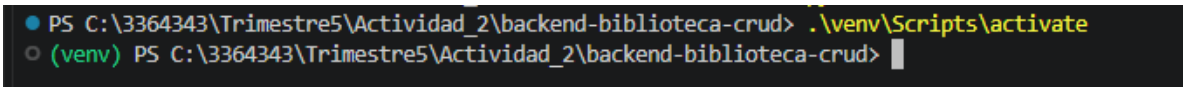


Verificación: Confirma que la ruta activa en tu terminal finalice en backend-biblioteca-crud.

Paso 2: Crear y activar el entorno virtual

Ejecuta los siguientes comandos para aislar las dependencias de Python:

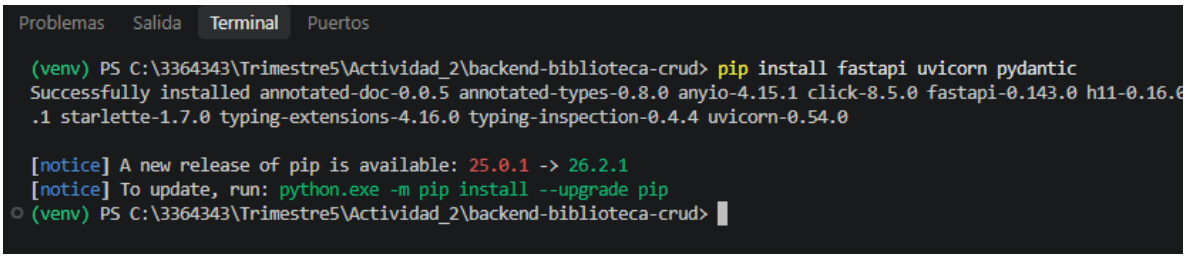




Verificación: Confirma que aparezca el prefijo (venv) al inicio de la línea de comandos en tu terminal.

Paso 3: Instalar FastAPI y Uvicorn

Instala el framework de desarrollo de APIs y el servidor ASGI con el comando:



Verificación: Ejecuta pip list y confirma que fastapi, uvicorn y pydantic figuren en la lista de paquetes instalados.

Paso 4: Escribir el código backend completo (main.py)

Crea un archivo llamado main.py dentro de la carpeta raíz backend-biblioteca-crud e incluye el siguiente código completo con las 5 operaciones CRUD y manejo de excepciones HTTP:

Explorador

backend-biblioteca-crud

venv

main.py

main.py

```python
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel
from typing import List, Optional

# Inicialización de la aplicación FastAPI
app = FastAPI(
    title="API Biblioteca Pública - ADSO",
    description="API RESTful completa con operaciones CRUD para auditoría de servicios web",
    version="2.0.0"
)

# Modelo de Datos con Pydantic para validación automática
class Libro(BaseModel):
    id: int
    titulo: str
    autor: str
    anio_publicacion: int
    disponible: bool = True

# Modelo auxiliar para la actualización parcial o total
class LibroUpdate(BaseModel):
    titulo: str
    autor: str
    anio_publicacion: int
    disponible: bool

# Base de datos simulada en memoria
db_libros: List[Libro] = [
    Libro(id=1, titulo="Cien Años de Soledad", autor="Gabriel García Márquez", anio_publicacion=1967, disponible=True),
    Libro(id=2, titulo="Don Quijote de la Mancha", autor="Miguel de Cervantes", anio_publicacion=1605, disponible=False)
]

# ---------------------------------------------------------------
# ENDPOINTS / RUTAS DE LA API (CRUD COMPLETO)
# ---------------------------------------------------------------

# 1. READ ALL (Obtener todos los libros)
@app.get("/api/v1/libros", response_model=List[Libro], status_code=status.HTTP_200_OK)
def obtener_todos_los_libros():
    return db_libros
```

Explorador

backend-biblioteca-crud

venv

main.py

Esquema

Línea De Tiempo

main.py > LibroUpdate

```python
# 2. READ ONE (Obtener un libro específico por ID)
@app.get("/api/v1/libros/{libro_id}", response_model=Libro, status_code=status.HTTP_200_OK)
def obtener_libro_por_id(libro_id: int):
    for libro in db_libros:
        if libro.id == libro_id:
            return libro
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"El libro con ID {libro_id} no existe en el sistema de la biblioteca."
    )


# 3. CREATE (Registrar un nuevo libro)
@app.post("/api/v1/libros", response_model=Libro, status_code=status.HTTP_201_CREATED)
def crear_nuevo_libro(libro: Libro):
    # Validar si el ID ya existe
    for l in db_libros:
        if l.id == libro.id:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Error de registro: Ya existe un libro con el ID {libro.id}."
            )
    db_libros.append(libro)
    return libro


# 4. UPDATE (Actualizar un libro existente - Método PUT)
@app.put("/api/v1/libros/{libro_id}", response_model=Libro, status_code=status.HTTP_200_OK)
def actualizar_libro(libro_id: int, datos_actualizados: LibroUpdate):
    for index, libro in enumerate(db_libros):
        if libro.id == libro_id:
            libro_modificado = Libro(
                id=libro_id,
                titulo=datos_actualizados.titulo,
                autor=datos_actualizados.autor,
                anio_publicacion=datos_actualizados.anio_publicacion,
                disponible=datos_actualizados.disponible
            )
            db_libros[index] = libro_modificado
            return libro_modificado

    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"No es posible actualizar. El libro con ID {libro_id} no fue encontrado."
    )
```

Explorador

backend-biblioteca-crud

venv

main.py

main.py > LibroUpdate

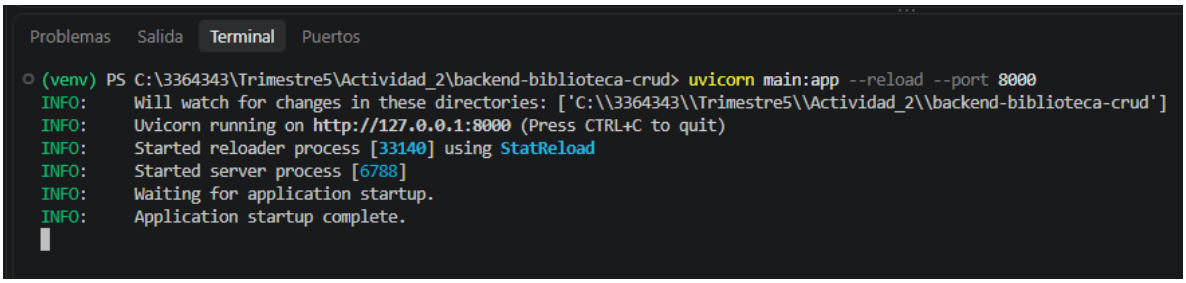
```python
# 5. DELETE (Eliminar un libro de la biblioteca)
@app.delete("/api/v1/libros/{libro_id}", status_code=status.HTTP_200_OK)
def eliminar_libro(libro_id: int):
    for index, libro in enumerate(db_libros):
        if libro.id == libro_id:
            db_libros.pop(index)
            return {"mensaje": f"El libro con ID {libro_id} fue eliminado exitosamente."}

    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"No es posible eliminar. El libro con ID {libro_id} no existe."
    )
```

Verificación: Confirma que el nombre main.py figure en la raíz de la carpeta del proyecto.

Paso 5: Iniciar el servidor backend

Inicia la aplicación ejecutando Uvicorn en la terminal:

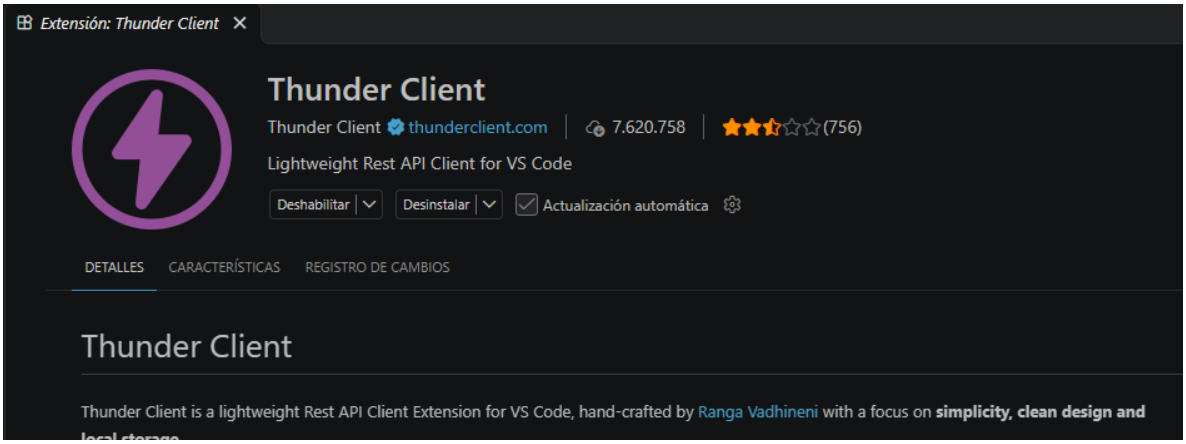


Importante: Asegúrate de que la terminal muestre el mensaje Application startup complete y Uvicorn running on http://127.0.0.1:8000 . No cierres esta terminal durante todo el laboratorio.

3. CONFIGURACIÓN Y EJECUCIÓN PASO A PASO EN THUNDER CLIENT

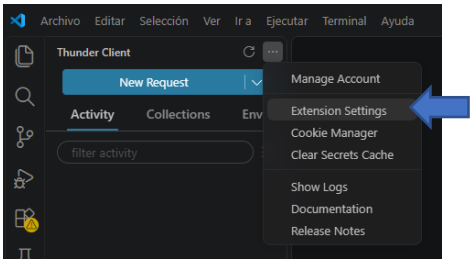
Paso 1: Instalar la extensión Thunder Client

En VS Code, haz clic en el ícono de **Extensiones** (Ctrl + Shift + X), busca Thunder Client e instala la extensión.

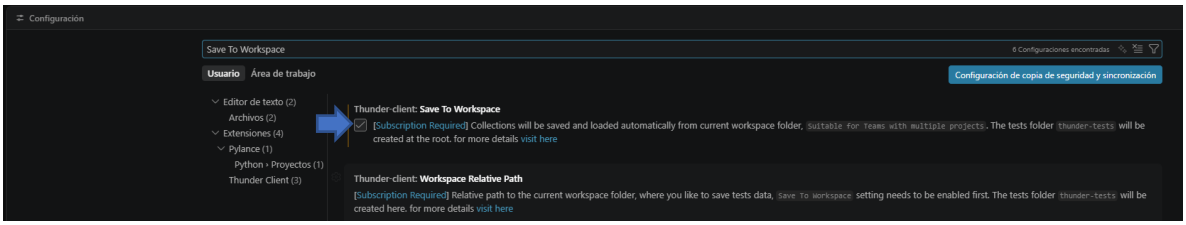


Paso 2: Habilitar el guardado de pruebas en el Workspace

1. Haz clic en el ícono del rayo (**Thunder Client**).
2. Haz clic en los **tres puntos (. . .)** ubicados en la parte superior derecha del panel de Thunder Client.
3. Selecciona **Extension Settings**.



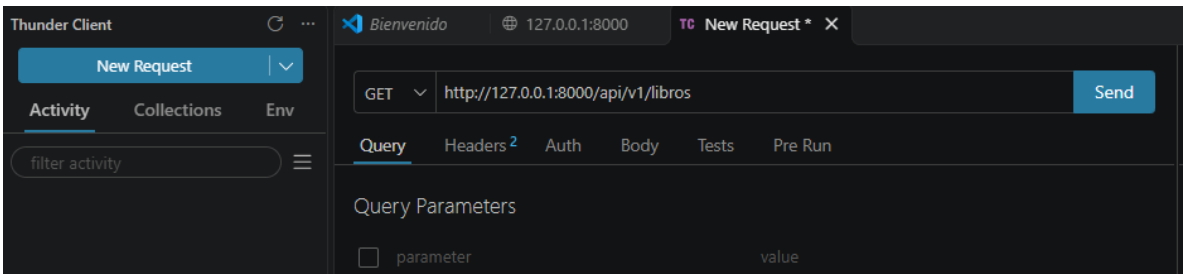
4. En el buscador de ajustes escribe: `Save To Workspace`.
5. **Marca la casilla de verificación** de la opción *Thunder Client: Save To Workspace*.



Verificación: Cierra la pestaña de ajustes; la configuración garantizará que tus peticiones se guarden en la carpeta local del proyecto.

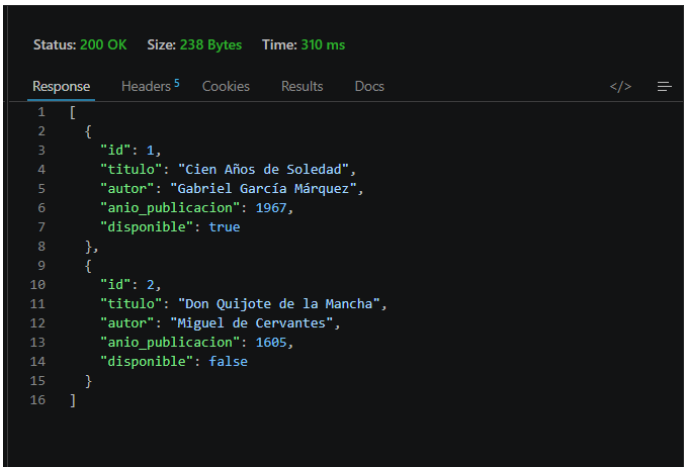
Paso 3: Prueba 1: Consultar la lista completa de libros (GET)

1. Haz clic en el botón azul **New Request**.
2. Verifica que el desplegable izquierdo de método esté seleccionado en **`GET`**.
3. En la casilla de dirección URL escribe exactamente:



4. Haz clic en el botón azul **Send**.

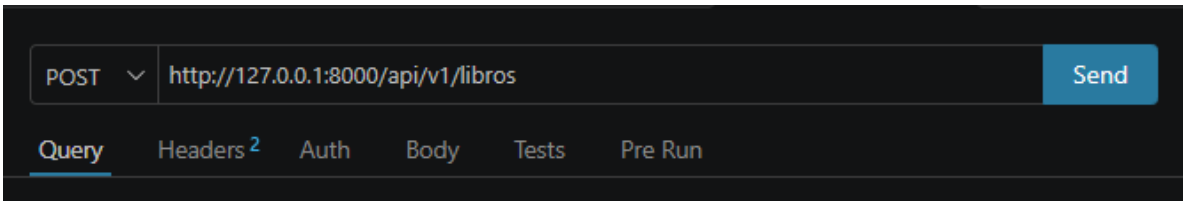
5. **Verificación de éxito:** Debe retornar un código verde `200 OK` y el cuerpo de respuesta (`Response`) mostrará la lista con los 2 libros iniciales.



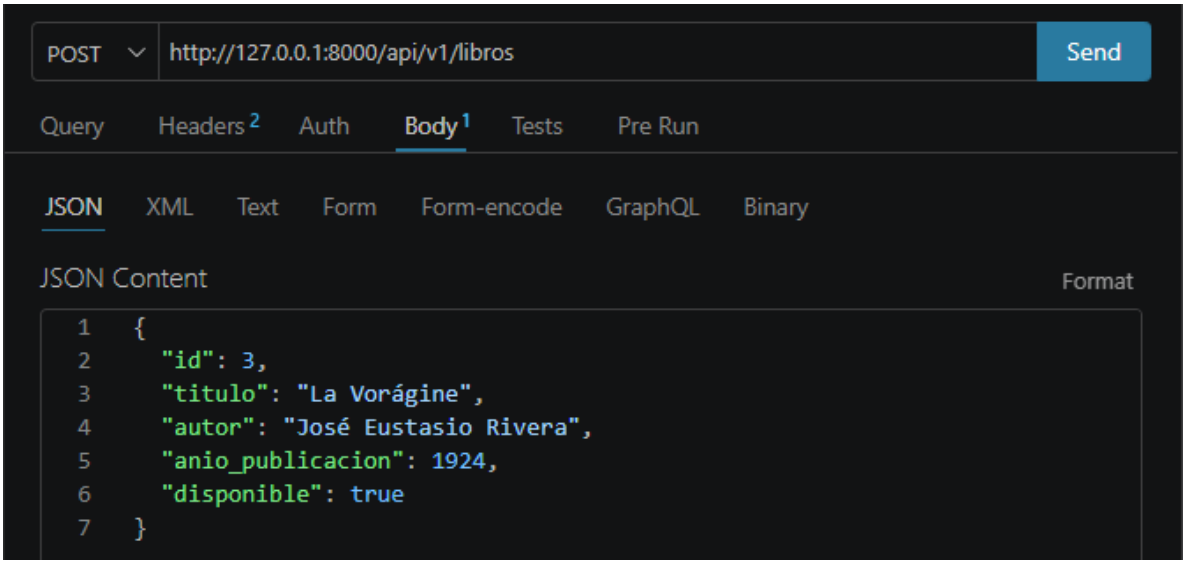
6. Haz clic en los tres puntos (. . .) de la pestaña de la petición, selecciona **Save Request** (o presiona `Ctrl + S`) y asígnale el nombre: `1. Listar Libros - 200 OK`.

Paso 4: Prueba 2: Registrar un nuevo libro (POST)

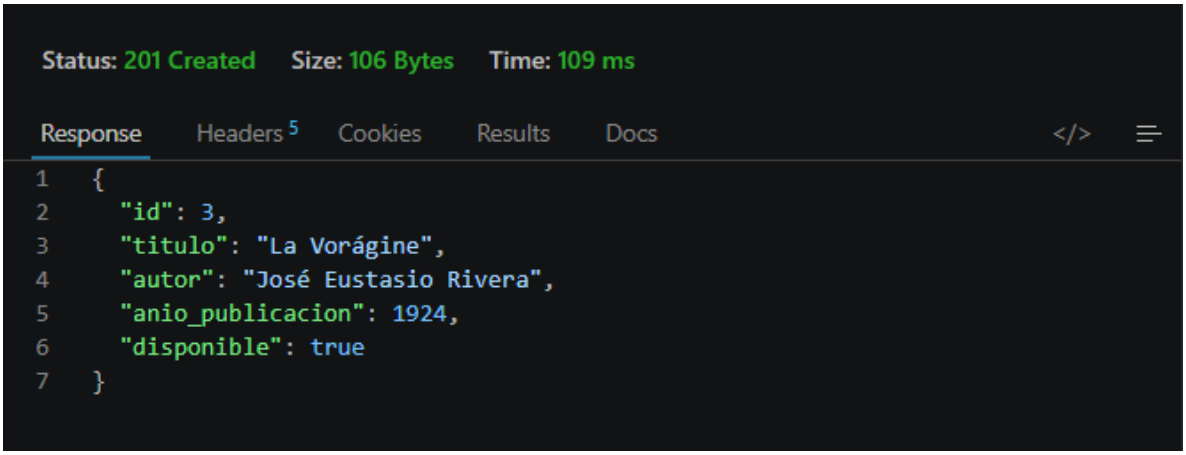
1. Haz clic en **New Request**.
2. Cambia el método HTTP de `GET` a **`POST`**.
3. Ingresa la URL:



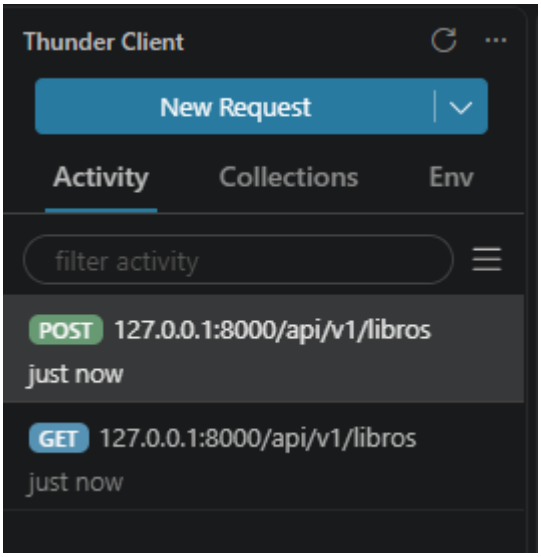
4. En el menú de opciones debajo de la URL, haz clic en la pestaña **Body** y luego selecciona **JSON**.
5. En el cuadro de texto pega la siguiente información:



6. Haz clic en **Send**.
7. **Verificación de éxito:** La respuesta debe ser **`201 Created`** mostrando el objeto creado con ID 3.

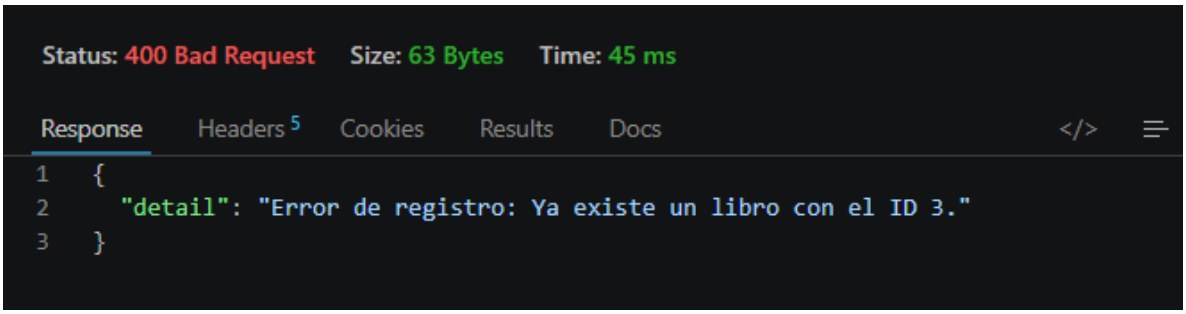


8. Presiona `Ctrl + S` y guarda la prueba como: `2. Crear Libro - 201 Created`.



Paso 5: Prueba 3: Auditar un error por libro duplicado (POST)

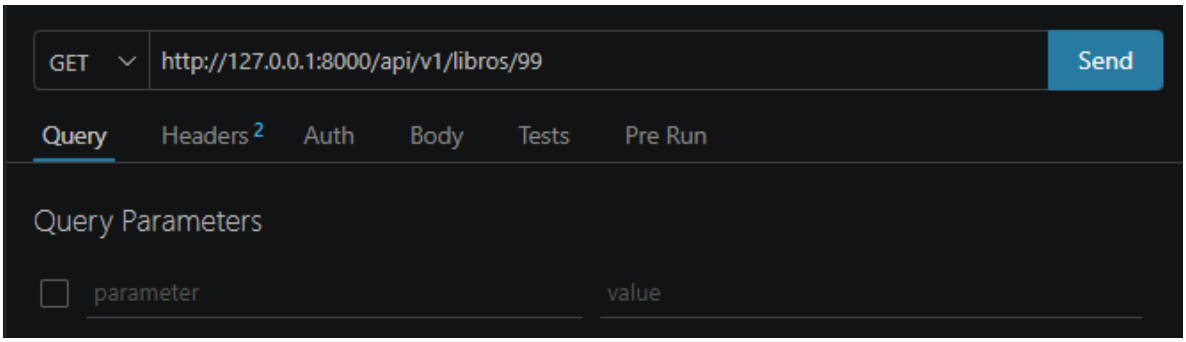
1. Mantente en la misma petición o abre un **New Request** con método **`POST`** y la URL
2. Mantén el mismo cuerpo JSON enviando nuevamente el `id: 3`.
3. Haz clic en **Send**.
4. **Verificación de éxito:** La API debe denegar el registro retornando un código naranja **`400 Bad Request`** con el mensaje de error: `"detail": "Error de registro: Ya existe un libro con el ID 3."`.



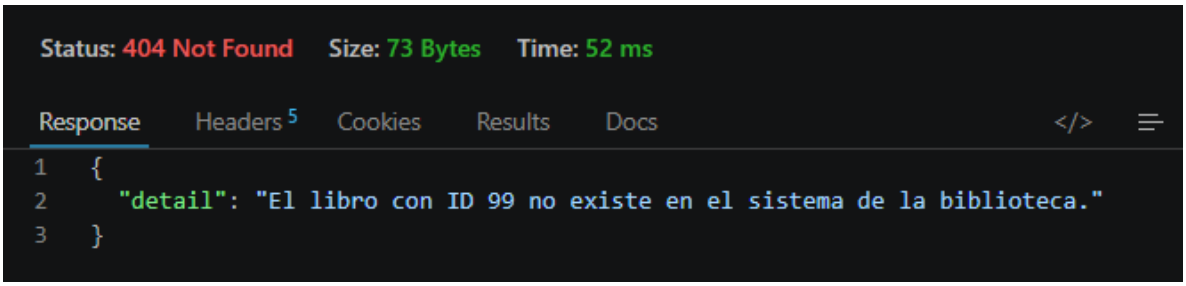
5. Presiona `Ctrl + S` y guarda la prueba como: `3. Crear Duplicado - 400 Bad Request`.

Paso 6: Prueba 4: Consultar un libro por ID inexistente (GET)

1. Haz clic en **New Request**.
2. Selecciona el método **`GET`**.
3. En la URL escribe:



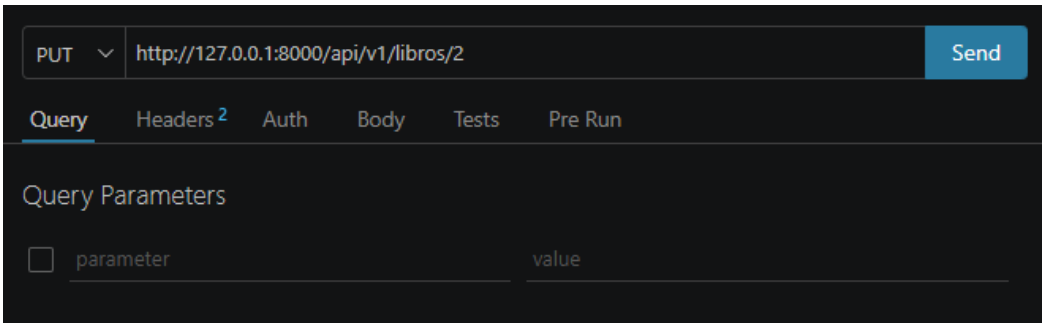
4. Haz clic en **Send**.
5. **Verificación de éxito:** La respuesta debe devolver un código **`404 Not Found`** con la advertencia: `"detail": "El libro con ID 99 no existe en el sistema de la biblioteca."`.



6. Presiona `Ctrl + S` y guarda la prueba como: `4. Consultar Inexistente - 404 Not Found`.

Paso 7: Prueba 5: Actualizar la información de un libro (PUT)

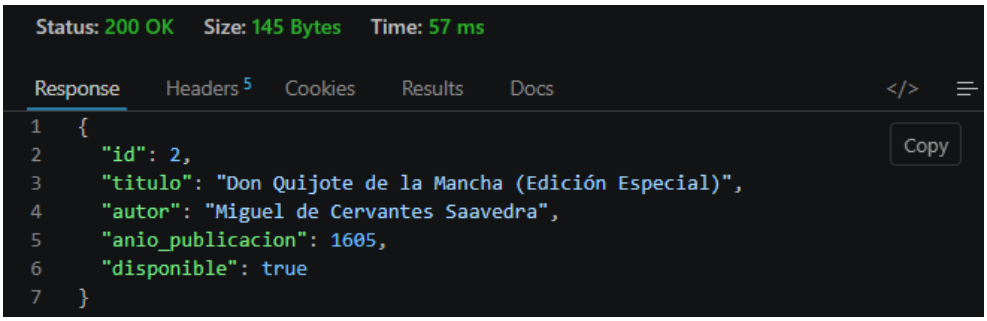
1. Haz clic en **New Request**.
2. Cambia el método HTTP a `PUT`.
3. En la URL escribe la ruta del libro número 2:



4. Ve a la pestaña **Body** $\rightarrow$ **JSON** y pega el objeto con la actualización deseada:

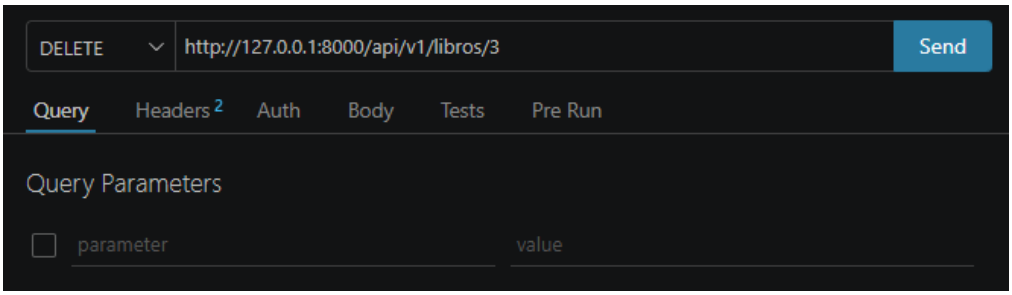


5. Haz clic en **Send**.
6. **Verificación de éxito:** La API debe retornar un código `200 OK` con el título y disponibilidad modificados exitosamente.
7. Presiona `Ctrl + S` y guarda la prueba como: `5. Actualizar Libro - 200 OK`.

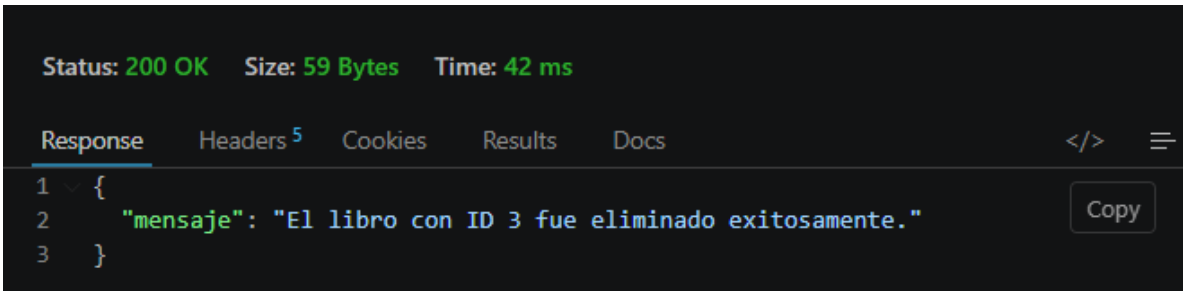


Paso 8: Prueba 6: Eliminar un libro de la biblioteca (DELETE)

1. Haz clic en **New Request**.
2. Selecciona el método HTTP **`DELETE`**.
3. En la URL escribe la ruta para eliminar el libro con ID 3:



4. Haz clic en **Send**.
5. **Verificación de éxito:** La respuesta debe retornar un código **`200 OK`** mostrando el mensaje: `"mensaje": "El libro con ID 3 fue eliminado exitosamente."`.



6. Presiona `Ctrl + S` y guarda la prueba como: `6. Eliminar Libro - 200 OK`.

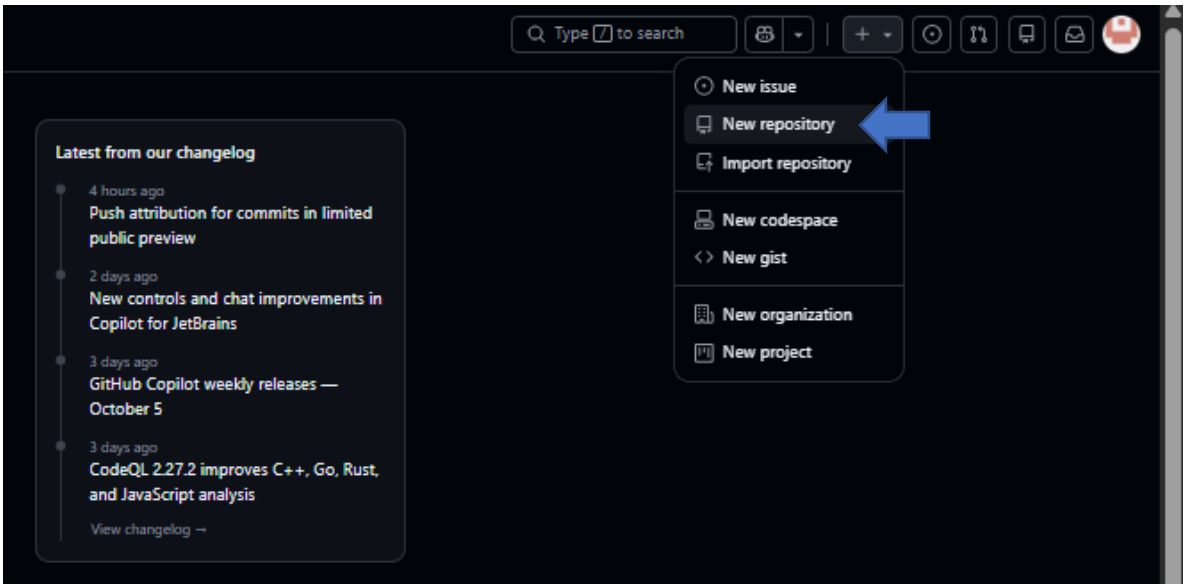
4. MATRIZ DE AUDITORÍA Y EVIDENCIAS DE CALIDAD

Como parte del informe técnico, los aprendices deberán completar y adjuntar la tabla de registro de auditoría con los resultados obtenidos durante la sesión:

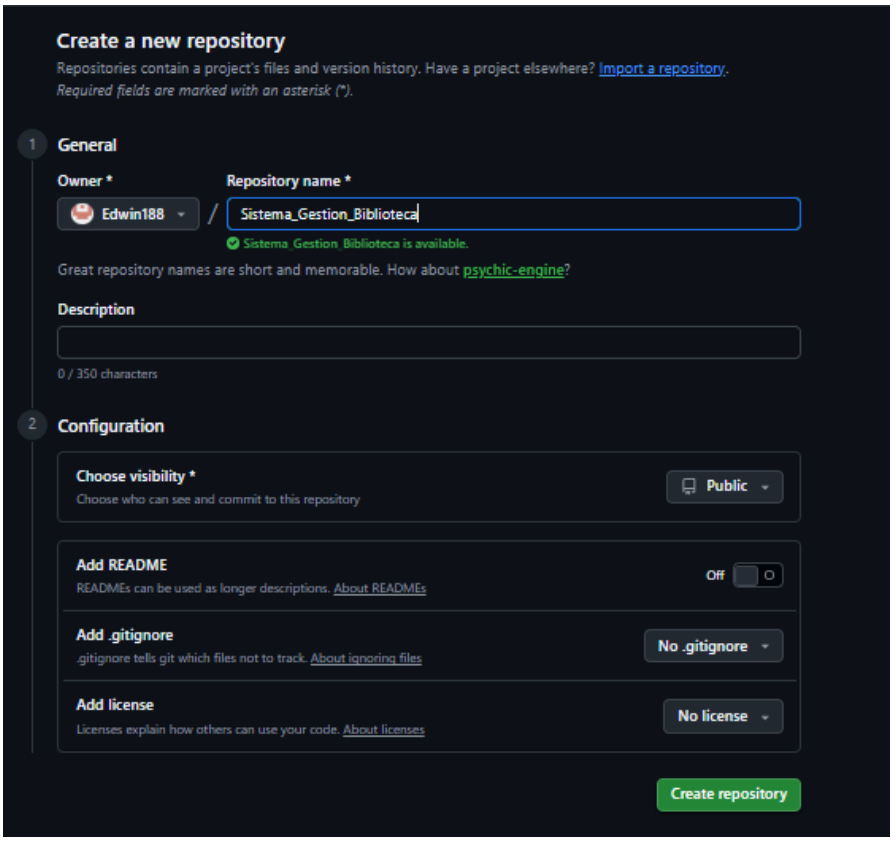
ID Prueba	Endpoint / Ruta	Método	Status Esperado	Status Obtenido	Latencia (ms)	Resultado
TC-01	/api/v1/libros	GET	200 OK	200 OK	15 ms	APROBADO
TC-02	/api/v1/libros	POST	201 Created	201 Created	22 ms	APROBADO
TC-03	/api/v1/libros	POST	400 Bad Request	400 Bad Request	10 ms	APROBADO
TC-04	/api/v1/libros/99	GET	404 Not Found	404 Not Found	8 ms	APROBADO
TC-05	/api/v1/libros/2	PUT	200 OK	200 OK	18 ms	APROBADO
TC-06	/api/v1/libros/3	DELETE	200 OK	200 OK	14 ms	APROBADO

Publicar el repositorio en GitHub

Abre la plataforma de GifHub y crea un nuevo repositorio



Colocamos el nombre del repositorio y clic en el boto de crear repositorio



Pasos

```
git init
git add .
git commit -m "feat: Se implementa API CRUD completa y suite de pruebas en Thunder Client AA2"
git branch -M main
git remote add origin https://github.com/TuUsuario/backend-biblioteca-crud.git
git push -u origin main
```

Proyectos asignados a los aprendices. Cada proyecto está ambientado en un sector productivo real y define una entidad principal con **5 campos clave** (incluyendo tipos de datos y restricciones de validación con Pydantic) para implementar el ciclo **CRUD completo en FastAPI** y auditarlo en **Thunder Client**:

1. Sistema de Droguería y Farmacia (Andrés Felipe Rodriguez)

- **Entidad principal:** `Medicamento`
- **Campos clave:**
    1. `id` (`int`): Identificador único del producto.
    2. `nombre` (`str`): Nombre comercial del medicamento.
    3. `principio_activo` (`str`): Componente farmacológico principal.
    4. `precio` (`float`): Valor de venta al público (debe ser mayor a 0).
    5. `requiere_formula` (`bool`): Indica si su venta exige receta médica.

2. Sistema de Gestión Clínica y Citas Médicas (Diego Fernando Garcia)

- **Entidad principal:** `Paciente`
- **Campos clave:**
    1. `id` (`int`): Identificador único del paciente.
    2. `documento_identidad` (`str`): Cédula o documento de identificación.
    3. `nombre_completo` (`str`): Nombres y apellidos del paciente.
    4. `eps` (`str`): Entidad promotora de salud afiliada.
    5. `edad` (`int`): Edad en años cumplidos (debe ser mayor a 0).

3. Sistema de Control de Gimnasio y Afiliaciones (Jeisy Valentina Hormanza)

- **Entidad principal:** `Membresia`
- **Campos clave:**
    1. `id` (`int`): Identificador del registro de afiliación.
    2. `codigo_usuario` (`str`): Código de carnet o documento del deportista.
    3. `tipo_plan` (`str`): Modalidad suscrita (Mensual, Trimestral, Anual).
    4. `valor_pagado` (`float`): Monto de la suscripción (debe ser mayor a 0).
    5. `estado_activo` (`bool`): Estado de la membresía para ingreso a sede.

4. Sistema de Comandos y Menú de Restaurante (Jhonatan Stiven Londoño)

- **Entidad principal:** `Plato`
- **Campos clave:**
    1. `id` (`int`): Identificador único del ítem del menú.
    2. `nombre_plato` (`str`): Denominación de la preparación culinaria.

3. `categoria` (`str`): Clasificación (Entrada, Fuerte, Postre, Bebida).
4. `precio_cop` (`float`): Precio en pesos colombianos (debe ser mayor a 0).
5. `apto_vegetarianos` (`bool`): Especificación dietética de la preparación.

5. Sistema de Gestión de Taller Automotriz (Julian Camilo Murillo)

- **Entidad principal:** `Vehiculo`
- **Campos clave:**
  1. `id` (`int`): Identificador del registro de ingreso a taller.
  2. `placa` (`str`): Matrícula vehicular única (ej. AAA123).
  3. `marca_modelo` (`str`): Marca y línea del vehículo.
  4. `kilometraje` (`int`): Lectura actual del odómetro (mayor o igual a 0).
  5. `orden_trabajo_activa` (`bool`): Indica si el vehículo está en mantenimiento.

6. Sistema de Tienda de Mascotas y Veterinaria (Kevin Eduardo garay)

- **Entidad principal:** `Mascota`
- **Campos clave:**
  1. `id` (`int`): Identificador de la historia clínica animal.
  2. `nombre_mascota` (`str`): Nombre del paciente animal.
  3. `especie` (`str`): Especie (Canino, Felino, Ave, Exótico).
  4. `peso_kg` (`float`): Peso registrado en kilogramos (debe ser mayor a 0).
  5. `vacunacion_al_dia` (`bool`): Estado del esquema de inmunización.

7. Sistema de Control de Ferretería y Materiales (Maikol Arley Vega)

- **Entidad principal:** `Herramienta`
- **Campos clave:**
  1. `id` (`int`): Identificador único del producto en inventario.
  2. `codigo_barras` (`str`): Referencia del fabricante o SKU.
  3. `descripcion` (`str`): Nombre y especificaciones del material.
  4. `stock_disponible` (`int`): Unidades físicas en bodega (mayor o igual a 0).
  5. `precio_unitario` (`float`): Valor de venta comercial (debe ser mayor a 0).

8. Sistema de Hotel y Reserva de Habitaciones (Maria Fernanda Espinel)

- **Entidad principal:** `Habitacion`
- **Campos clave:**
  1. `id` (`int`): Identificador o número de habitación.
  2. `tipo_habitacion` (`str`): Categórica (Sencilla, Doble, Suite, Familiar).
  3. `tarifa_noche` (`float`): Costo por noche de hospedaje (debe ser mayor a 0).
  4. `capacidad_personas` (`int`): Límite de huéspedes autorizados (mínimo 1).
  5. `disponible_reserva` (`bool`): Estado de ocupación o mantenimiento.

9. Sistema de Inmobiliaria y Arrendamiento (Mariangel Liseth Mocharrafich)

- **Entidad principal:** `Inmueble`
- **Campos clave:**
    1. `id` (`int`): Identificador único del predio.
    2. `direccion` (`str`): Ubicación geográfica del inmueble.
    3. `canon_arriendo` (`float`): Mensualidad de arrendamiento (debe ser mayor a 0).
    4. `numero_alcobas` (`int`): Cantidad de habitaciones construidas (mínimo 1).
    5. `incluye_administracion` (`bool`): Especificación de costos comunes.

10. Sistema de Supermercado y Puntos de Venta (Sharitk Lorena Toro)

- **Entidad principal:** `ProductoAbarrotes`
- **Campos clave:**
    1. `id` (`int`): Identificador del artículo en caja.
    2. `nombre_producto` (`str`): Descripción del vívere o bien de consumo.
    3. `marca` (`str`): Empresa fabricante o distribuidora.
    4. `precio_venta` (`float`): Precio en góndola (debe ser mayor a 0).
    5. `es_perecedero` (`bool`): Indica si requiere fecha de vencimiento próxima.

11. Sistema de Lavandería y Servicio Técnico (Emmanuel David Jimenez)

- **Entidad principal:** `PrendaServicio`
- **Campos clave:**
    1. `id` (`int`): Código de orden de recepción.
    2. `cliente_propietario` (`str`): Nombre del usuario que solicita el servicio.
    3. `tipo_prenda` (`str`): Clasificación (Traje, Cobija, Calzado, Uniforme).
    4. `costo_lavado` (`float`): Valor cobrado por la limpieza (debe ser mayor a 0).
    5. `listo_para_entrega` (`bool`): Estado del proceso operativo.

12. Sistema de Control de Parqueaderos y Estacionamiento (Skarleth Daian Aya)

- **Entidad principal:** `TicketParqueadero`
- **Campos clave:**
    1. `id` (`int`): Folio o consecutivo del tiquete.
    2. `placa_vehiculo` (`str`): Identificador del automóvil o motocicleta.
    3. `tipo_vehiculo` (`str`): Categoría (Carro, Moto, Bicicleta).
    4. `hora_ingreso` (`str`): Hora de registro de entrada (formato HH:MM).
    5. `pago_realizado` (`bool`): Estado de cancelación de la tarifa.

13. Sistema de Agencia de Viajes y Paquetes Turísticos (Gabriel Felipe Daza)

- **Entidad principal:** `PaqueteTuristico`
- **Campos clave:**
    1. `id` (`int`): Identificador del plan vacacional.
    2. `destino` (`str`): Ciudad o región geográfica del viaje.
    3. `duracion_dias` (`int`): Días totales de estancia (mínimo 1).
    4. `precio_por_persona` (`float`): Tarifa individual (debe ser mayor a 0).
    5. `incluye_tiquetes_aereos` (`bool`): Especificación del transporte incluido.

14. Sistema de Control de Barbería y Peluquería (Brayan Alexander Cotrina)

- **Entidad principal:** `CitaBarberia`
- **Campos clave:**
    1. `id` (`int`): Identificador del turno o reserva.
    2. `nombre_cliente` (`str`): Nombre del usuario agendado.
    3. `servicio_solicitado` (`str`): Tipo de atención (Corte, Barba, Combo).
    4. `precio_servicio` (`float`): Valor del trabajo estilístico (debe ser mayor a 0).
    5. `atendido` (`bool`): Confirmación del cumplimiento del turno.

15. Sistema de Gestión de Tienda de Ropa y Calzado (Dilan Mateo Peñaloza)

- **Entidad principal:** `ArticuloModa`
- **Campos clave:**
    1. `id` (`int`): Identificador único de la prenda.
    2. `referencia_sku` (`str`): Código alfanumérico del catálogo.
    3. `talla` (`str`): Tamaño o medida (XS, S, M, L, XL, 38, 40).
    4. `precio_publico` (`float`): Valor comercial en etiqueta (debe ser mayor a 0).
    5. `en_promocion` (`bool`): Indica si aplica descuento de temporada.

16. Sistema de Centro Educativo y Cursos de Capacitación (Niko Julian Ardila)

- **Entidad principal:** `Curso`
- **Campos clave:**
    1. `id` (`int`): Identificador de la ficha del curso.
    2. `nombre_curso` (`str`): Denominación del programa de formación.
    3. `duracion_horas` (`int`): Intensidad horaria asignada (debe ser mayor a 0).
    4. `costo_inscripcion` (`float`): Matrícula o valor del taller (debe ser mayor a 0).
    5. `cupo_abierto` (`bool`): Disponibilidad de inscripciones en el sistema.